LAPORAN PRAKTIKUM

STRUKTUR DATA

“Single Linked List”

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

M Galid A

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur penulis ucapkan kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data dengan materi Single Linked List ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas praktikum yang telah dilakukan pada mata kuliah Struktur Data. Pada praktikum ini, penulis mempelajari konsep dasar Single Linked List, cara kerja node dan pointer, serta implementasi operasi-operasi dasar seperti penambahan, penghapusan, dan penelusuran data pada linked list.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

Penulis juga mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah membantu selama proses praktikum maupun penyusunan laporan ini.

Akhir kata, penulis berharap laporan praktikum ini dapat bermanfaat serta menambah pemahaman mengenai konsep Single Linked List dalam struktur data.

Wassalamu'alaikum Warahmatullahi Wabarakatuh

Padang, 6 Mei 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Single Linked List 3
2.2 Operasi-Operasi yang ada pada Linked List 3
2.3 Impementasi Single Linked List pada Praktikum 4
2.3.1 Kode Pertama(NodeSLL_2511531013) 4
2.3.2 Kode Kedua(TambahSLL_2511531013) 6
2.3.3 Kode Ketiga(PencarianSLL_2511531013) 8
2.3.4 Kode Keempat(HapusSLL_2511531013) 10
BAB III PENUTUP 12
3.1 Kesimpulan 12
3.2 Saran 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengolahan data yang terstruktur sangat diperlukan agar suatu program dapat berjalan dengan lebih efektif dan efisien. Oleh karena itu, diperlukan pemahaman mengenai struktur data sebagai dasar dalam menyimpan, mengatur, dan mengelola data di dalam program. Salah satu struktur data yang sering digunakan adalah Linked List.

Single Linked List merupakan struktur data linear yang terdiri dari beberapa node yang saling terhubung. Setiap node memiliki data dan sebuah link yang menunjuk ke node berikutnya. Berbeda dengan array yang memiliki ukuran tetap, Single Linked List bersifat dinamis sehingga penambahan dan penghapusan data dapat dilakukan dengan lebih fleksibel tanpa harus memindahkan seluruh elemen data.

Pada praktikum ini, dilakukan implementasi Single Linked List menggunakan bahasa pemrograman Java. Praktikum berfokus pada operasi-operasi dasar pada Single Linked List, seperti menambahkan simpul di bagian depan, belakang, dan posisi tertentu, serta menghapus simpul pada bagian awal, akhir, maupun posisi tertentu dalam linked list.

Selain itu, praktikum ini juga membantu mahasiswa memahami cara kerja node, proses traversal, serta hubungan antar simpul dalam struktur Single Linked List. Dengan memahami implementasi dasar ini, mahasiswa diharapkan mampu menerapkan konsep Linked List dalam penyelesaian berbagai permasalahan pemrograman yang berkaitan dengan pengolahan data dinamis.

1.2 Tujuan

Adapun tujuan dari praktikum Struktur Data dengan materi Single Linked List ini adalah sebagai berikut:

1. Mahasiswa mampu memahami konsep dasar Single Linked List sebagai salah satu struktur data linear dinamis.
2. Mahasiswa mampu memahami fungsi node dan pointer dalam menghubungkan data pada Single Linked List.
3. Mahasiswa mampu mengimplementasikan operasi dasar Single Linked List menggunakan bahasa pemrograman Java.
4. Mahasiswa mampu membuat proses penambahan data pada Single Linked List, baik di bagian depan, belakang, maupun posisi tertentu.
5. Mahasiswa mampu melakukan proses penghapusan node pada Single Linked List, seperti menghapus head, node terakhir, dan node pada posisi tertentu.

1.3 Manfaat

Adapun manfaat dari praktikum Struktur Data dengan materi Single Linked List ini adalah sebagai berikut:

1. Mahasiswa dapat memahami cara kerja struktur data dinamis dalam pengelolaan data pada program.
2. Mahasiswa dapat melatih kemampuan logika pemrograman dalam proses penambahan dan penghapusan data pada linked list.
3. Mahasiswa dapat memahami hubungan antar node pada Single Linked List melalui penggunaan pointer atau reference.
4. Mahasiswa dapat meningkatkan kemampuan implementasi struktur data menggunakan bahasa pemrograman Java.
5. Mahasiswa dapat menjadikan materi Single Linked List sebagai dasar untuk mempelajari struktur data lain yang lebih kompleks, seperti Double Linked List, Stack, Queue, dan Tree.

BAB II
PEMBAHASAN

2.1 Pengertian Single Linked List

Linked List merupakan koleksi linear dari data, yang disebut sebagai nodes, dimana setiap node akan menunjuk pada node lain melalui sebuah pointer. Linked List dapat didefinisikan pula sebagai kumpulan nodes yang merepresentasikan sebuah sequence. Sebuah linked list yang hanya memiliki 1 penghubung ke node lain disebut sebagai single linked list.

Di dalam sebuah linked list, ada 1 pointer yang menjadi gambaran besar, yakni pointer HEAD yang menunjuk pada node pertama di dalam linked list itu sendiri. Sebuah linked list dikatakan kosong apabila isi pointer head adalah NULL.

2.2 Operasi-Operasi yang ada pada Linked List

1. Insert

   Istilah Insert berarti menambahkan sebuah simpul baru ke dalam suatu linked list.Stack yang Diimplementasikan Menggunakan Linked List
2. Is Empty

   Fungsi ini menentukan apakah linked list kosong atau tidak.Circular Stack
3. Find First

   Fungsi ini mencari elemen pertama dari linked list
4. Find Next

   Fungsi ini mencari elemen sesudah elemen yang ditunjuk now
5. Retrieve

   Fungsi ini mengambil elemen yang ditunjuk oleh now.Elemen tersebut lalu dikembalikan oleh fungsi.
6. Update

Fungsi ini mengubah elemen yang ditunjuk oleh now dengan isi dari sesuatu

7. Delete Now

   Fungsi ini menghapus elemen yang ditunjuk oleh now. Jika yang dihapus adalah elemen pertama dari linked list (head), head akan berpindah ke elemen berikut.

8. Delete Head

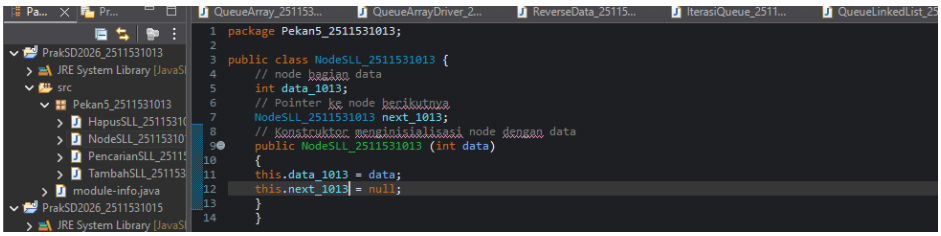
   Fungsi ini menghapus elemen yang ditunjuk head.Head berpindah ke elemen sesudahnya.

9. Clear

   Fungsi ini menghapus linked list yang sudah ada. Fungsi ini wajib dilakukan bila anda ingin mengakhiri program yang menggunakan linked list. Jika anda melakukannya, data-data yang dialokasikan ke memori pada program sebelumnya akan tetap tertinggal di dalam memori.

2.3 Impementasi Single Linked List pada Praktikum

2.3.1 Kode Pertama(NodeSLL_2511531013)



```java
package Pekan5_2511531013;

public class NodeSLL_2511531013 {
    // node bagian data
    int data_1013;
    // Pointer ke node berikutnya
    NodeSLL_2511531013 next_1013;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511531013 (int data)
    {
    this.data_1013 = data;
    this.next_1013 = null;
    }
    }
```

Kode Program 2.1

Program NodeSLL_2511531013 merupakan kelas yang digunakan untuk membuat node pada struktur data Single Linked List. Node berfungsi sebagai tempat penyimpanan data sekaligus penghubung ke node berikutnya.

Pada program ini terdapat atribut data_1013 yang bertipe int dan digunakan untuk menyimpan data pada node. Selain itu, terdapat

atribut next_1013 yang bertipe NodeSLL_2511531013 dan berfungsi sebagai pointer atau penunjuk ke node berikutnya dalam Linked List.

Konstruktor NodeSLL_2511531013(int data) digunakan untuk menginisialisasi node baru. Nilai parameter data akan disimpan ke dalam atribut data_1013 menggunakan keyword this. Sedangkan atribut next_1013 diatur bernilai null, yang menandakan bahwa node tersebut belum terhubung dengan node lain.

Program ini menjadi dasar dalam pembuatan struktur data Single Linked List, karena setiap node nantinya akan saling terhubung melalui pointer next_1013.

2.3.2 Kode Kedua(TambahSLL_2511531013)

```java
package Pekan5_2511531013;

public class TambahSLL_2511531013 {

    // tambah di depan
    public static NodeSLL_2511531013 insertAtFront(NodeSLL_2511531013 head_1013, int value_1013) {
        NodeSLL_2511531013 new_node_1013 = new NodeSLL_2511531013(value_1013);
        new_node_1013.next_1013 = head_1013;
        return new_node_1013;
    }

    // tambah di akhir
    public static NodeSLL_2511531013 insertAtEnd(NodeSLL_2511531013 head_1013, int value_1013) {
        NodeSLL_2511531013 newNode_1013 = new NodeSLL_2511531013(value_1013);

        if (head_1013 == null) {
            return newNode_1013;
        }

        NodeSLL_2511531013 last_1013 = head_1013;

        while (last_1013.next_1013 != null) {
            last_1013 = last_1013.next_1013;
        }

        last_1013.next_1013 = newNode_1013;
        return head_1013;
    }

    static NodeSLL_2511531013 getNode(int data_1013) {
        return new NodeSLL_2511531013(data_1013);
    }

    static NodeSLL_2511531013 insertPos(NodeSLL_2511531013 headNode_1013, int position_1013, int value_1013) {

        NodeSLL_2511531013 head_1013 = headNode_1013;

        if (position_1013 < 1) {
            System.out.println("Invalid position");
            return head_1013;
        }

        if (position_1013 == 1) {
            NodeSLL_2511531013 new_node_1013 = new NodeSLL_2511531013(value_1013);
            new_node_1013.next_1013 = head_1013;
            return new_node_1013;
        } else {
            while (position_1013-- != 0) {

                if (position_1013 == 1) {
                    NodeSLL_2511531013 newNode_1013 = getNode(value_1013);
                    newNode_1013.next_1013 = headNode_1013.next_1013;
                    headNode_1013.next_1013 = newNode_1013;
                    break;
                }

                headNode_1013 = headNode_1013.next_1013;

                if (headNode_1013 == null) {
                    System.out.println("Posisi di luar jangkauan");
                    break;
                }
            }
        }
        return head_1013;
    }

    public static void printList_1013(NodeSLL_2511531013 head_1013) {
        NodeSLL_2511531013 curr_1013 = head_1013;

        while (curr_1013 != null) {
            if (curr_1013.next_1013 != null) {
                System.out.print(curr_1013.data_1013 + " --> ");
            } else {
                System.out.print(curr_1013.data_1013);
            }
            curr_1013 = curr_1013.next_1013;
        }
        System.out.println();
    }

    public static void main(String[] args) {

        // buat linked list 2->3->5->6
        NodeSLL_2511531013 head_1013 = new NodeSLL_2511531013(2);

        head_1013.next_1013 = new NodeSLL_2511531013(3);
        head_1013.next_1013.next_1013 = new NodeSLL_2511531013(5);
        head_1013.next_1013.next_1013.next_1013 = new NodeSLL_2511531013(6);

        // cetak list awal
        System.out.print("Senarai berantai awal: ");
        printList_1013(head_1013);

        // tambah di depan
        System.out.print("Tambah 1 simpul di depan: ");
        int data_1013 = 1;
        head_1013 = insertAtFront(head_1013, data_1013);
        printList_1013(head_1013);

        // tambah di belakang
        System.out.print("Tambah 1 simpul di belakang: ");
        int data2_1013 = 7;
        head_1013 = insertAtEnd(head_1013, data2_1013);
        printList_1013(head_1013);

        // tambah di posisi tertentu
        System.out.print("Tambah 1 simpul ke data 4: ");
        int data3_1013 = 4;
        int pos_1013 = 4;
         head_1013 = insertPos(head_1013, pos_1013, data3_1013);
         printList_1013(head_1013);
    }
}
```

Kode Program 2.2

Program TambahSLL_2511531013 digunakan untuk melakukan penambahan node pada struktur data Single Linked List.

Program ini memiliki beberapa operasi penambahan, yaitu menambah node di depan, menambah node di akhir, dan menambah node pada posisi tertentu.

Method insertAtFront() digunakan untuk menambahkan node baru di bagian depan Linked List. Node baru dibuat terlebih dahulu, lalu pointer next_1013 dari node baru diarahkan ke head_1013 lama. Setelah itu, node baru dikembalikan sebagai head_1013 yang baru.

Method insertAtEnd() digunakan untuk menambahkan node baru di bagian akhir Linked List. Jika head_1013 bernilai null, maka node baru langsung menjadi head. Jika tidak kosong, program akan menelusuri node satu per satu sampai menemukan node terakhir. Setelah node terakhir ditemukan, pointer next_1013 diarahkan ke node baru.

Method getNode() digunakan untuk membuat node baru berdasarkan data yang diberikan. Method ini membantu proses pembuatan node agar lebih sederhana.

Method insertPos() digunakan untuk menambahkan node pada posisi tertentu. Jika posisi kurang dari 1, maka program akan menampilkan pesan bahwa posisi tidak valid. Jika posisi yang dimasukkan adalah 1, maka node baru akan ditambahkan di bagian depan. Namun, jika posisi lebih dari 1, program akan menelusuri Linked List sampai posisi yang diinginkan, lalu menyisipkan node baru di posisi tersebut. Jika posisi berada di luar jangkauan, maka program akan menampilkan pesan bahwa posisi tidak tersedia.

Method printList_1013() digunakan untuk menampilkan seluruh isi Linked List. Program akan menelusuri setiap node dari awal sampai akhir, kemudian mencetak nilai data dengan tanda panah → sebagai penghubung antar node.

Pada method main, pertama dibuat Linked List awal dengan isi data 2 -> 3 -> 5 -> 6. Setelah itu, program menampilkan isi list

awal. Kemudian dilakukan penambahan data 1 di bagian depan, sehingga data menjadi 1 -> 2 -> 3 -> 5 -> 6. Selanjutnya ditambahkan data 7 di bagian belakang, sehingga data menjadi 1 -> 2 -> 3 -> 5 -> 6 -> 7. Terakhir, program menambahkan data 4 pada posisi ke-4, sehingga data menjadi 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7.

2.3.3 Kode Ketiga(PencarianSLL_2511531013)

```java
package Pekan5_2511531013;

public class PencarianSLL_2511531013 {
    static boolean searchKey (NodeSLL_2511531013 head, int Key) {
        NodeSLL_2511531013 curr_1013= head;
        while (curr_1013 != null) {
            if(curr_1013.data_1013==Key)
                return true;
            curr_1013=curr_1013.next_1013; }
        return false; }
    public static void traversal (NodeSLL_2511531013 head) {
        //mulai dari head
        NodeSLL_2511531013 curr_1013=head;
        //telusuri sampai pointer null
        while (curr_1013 != null) {
            System.out.print(" " + curr_1013.data_1013);
            curr_1013 = curr_1013.next_1013;}
        System.out.println(); }

        public static void main (String[] args) {
            NodeSLL_2511531013 head_1013 = new NodeSLL_2511531013(14);
            head_1013.next_1013 = new NodeSLL_2511531013(21);
            head_1013.next_1013.next_1013 = new NodeSLL_2511531013(13);
            head_1013.next_1013.next_1013.next_1013 = new NodeSLL_2511531013(30);
            head_1013.next_1013.next_1013.next_1013.next_1013= new NodeSLL_2511531013(10);
            System.out.print("Penelesuran SLL : ");
            traversal (head_1013);
            //data yang akan dicari
            int Key=30;
            System.out.print("cari data " +Key+ " = ");
            if (searchKey(head_1013, Key))
                System.out.println("Ketemu");
            else
                System.out.println("Tidak ada");
        }

        }
```

Kode Program 2.3

Program PencarianSLL_2511531013 digunakan untuk melakukan pencarian data pada struktur data *Single Linked List*. Program ini juga menampilkan seluruh isi *Linked List* menggunakan proses traversal.

Method searchKey() digunakan untuk mencari data tertentu pada *Linked List*. Method ini menerima parameter head sebagai node awal dan Key sebagai data yang ingin dicari. Proses pencarian dilakukan menggunakan pointer curr_1013 yang mulai dari node pertama. Program akan menelusuri node satu per satu menggunakan perulangan while sampai mencapai node terakhir. Jika data pada node sama dengan nilai Key, maka method akan mengembalikan nilai true.

Namun jika data tidak ditemukan sampai akhir list, method akan mengembalikan nilai false.

Method traversal() digunakan untuk menampilkan seluruh isi *Single Linked List*. Pointer curr_1013 digunakan untuk menelusuri node mulai dari head hingga bernilai null. Setiap data pada node akan dicetak ke layar secara berurutan.

Pada method main, pertama dibuat sebuah *Single Linked List* dengan data 14 -> 21 -> 13 -> 30 -> 10. Setelah itu program menampilkan isi list menggunakan method traversal().

Selanjutnya program melakukan pencarian data dengan nilai 30. Method searchKey() dipanggil untuk mengecek apakah data tersebut terdapat di dalam *Linked List*. Karena data 30 ada di dalam list, maka program akan menampilkan pesan “Ketemu”. Jika data tidak ada, maka program akan menampilkan pesan “Tidak ada”.

2.3.4 Kode Keempat(HapusSLL_2511531013)

```java
package Pekan5_2511531013;
public class HapusSLL_2511531013 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511531013 deleteHead(NodeSLL_2511531013 head) {
        // jika SLL kosong
        if (head == null)
            return null;
        // pindahkan head ke node berikutnya
        head = head.next_1013;
        // Return head baru
        return head;
    }
    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511531013 removeLastNode(NodeSLL_2511531013 head) {
        // jika list kosong, return null
        if (head == null) {
            return null;
        }
        //jika list satu node, hapus node dan return null
        if (head.next_1013 == null) {
            return null;
        }
        //temukan node terakhir ke dua
        NodeSLL_2511531013 secondLast = head;
        while (secondLast.next_1013.next_1013 != null) {
            secondLast = secondLast.next_1013;
        }
        // hapus node terakhir
        secondLast.next_1013 = null;
        return head;
    }
    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511531013 deleteNode(NodeSLL_2511531013 head, int position) {
        NodeSLL_2511531013 temp_1013 = head;
        NodeSLL_2511531013 prev_1013 = null;
        // jika linked list null
        if (temp_1013 == null)
            return head;
        // kasus 1: head dihapus
        if (position == 1) {
            head = temp_1013.next_1013;
            return head;    }
        // kasus 2: menghapus node di tengah
        // telusuri ke node yang dihapus
        for (int i_1013 = 1; temp_1013 != null && i_1013 < position; i_1013++) {
            prev_1013 = temp_1013;
            temp_1013 = temp_1013.next_1013;   }
        // jika ditemukan, hapus node
        if (temp_1013 != null) {
            prev_1013.next_1013 = temp_1013.next_1013;
        } else {
            System.out.println("Data tidak ada");        }
        return head;    }
    // fungsi mencetak SLL
    public static void printList(NodeSLL_2511531013 head) {
        NodeSLL_2511531013 curr_1013 = head;
        while (curr_1013.next_1013 != null) {
            System.out.print(curr_1013.data_1013+"-->");
            curr_1013 = curr_1013.next_1013;       }
        if (curr_1013.next_1013==null) {
            System.out.print(curr_1013.data_1013);    }
        System.out.println(); }
    //kelas main
    public static void main(String[] args) {
        // buat SLL 1 -> 2 -> 3 -> 4 -> 5-> 6 -> null
        NodeSLL_2511531013 head_1013 = new NodeSLL_2511531013(1);
        head_1013.next_1013 = new NodeSLL_2511531013(2);
        head_1013.next_1013.next_1013 = new NodeSLL_2511531013(3);
        head_1013.next_1013.next_1013.next_1013 = new NodeSLL_2511531013(4);
        head_1013.next_1013.next_1013.next_1013.next_1013 = new NodeSLL_2511531013(5);
        head_1013.next_1013.next_1013.next_1013.next_1013.next_1013 = new NodeSLL_2511531013(6);
        // cetak list awal
        System.out.println("list awal: ");
        printList(head_1013);
        // hapus head
        head_1013 = deleteHead(head_1013);
        System.out.println("List setelah head dihapus: ");
        printList(head_1013);
        //hapus node terakhir
        head_1013 = removeLastNode(head_1013);
        System.out.println("List setelah simpul terakhir di hapus: ");
        printList(head_1013);
        // Deleting node at position 2
        int position = 2;
        head_1013 = deleteNode(head_1013, position);
        // Print list after deletion
        System.out.println("List setelah posisi 2 dihapus: ");
        printList(head_1013);
    }

}
```

Kode Program 2.4

Program HapusSLL_2511531013 digunakan untuk melakukan penghapusan node pada struktur data Single Linked List. Program ini memiliki beberapa operasi penghapusan, yaitu menghapus node pertama, menghapus node terakhir, dan menghapus node pada posisi tertentu.

Method deleteHead() digunakan untuk menghapus node pertama atau head pada Linked List. Jika list kosong, maka method akan mengembalikan nilai null. Jika list tidak kosong, maka posisi head akan dipindahkan ke node berikutnya.

Method removeLastNode() digunakan untuk menghapus node terakhir pada Linked List. Jika list kosong atau hanya memiliki satu node, maka method akan mengembalikan null. Jika node lebih dari satu, program akan menelusuri list sampai menemukan node sebelum node terakhir, lalu pointer next_1013 pada node tersebut diubah menjadi null.

Method deleteNode() digunakan untuk menghapus node pada posisi tertentu. Jika posisi yang dihapus adalah posisi pertama, maka head akan dipindahkan ke node berikutnya. Jika posisi berada di tengah atau akhir list, program akan menelusuri node sampai posisi yang ditentukan. Setelah node ditemukan, pointer dari node sebelumnya akan diarahkan ke node setelah node yang dihapus. Jika posisi tidak ditemukan, maka program akan menampilkan pesan "Data tidak ada".

Method printList() digunakan untuk menampilkan isi Linked List dari awal sampai akhir. Setiap node dicetak secara berurutan dengan tanda panah → sebagai penghubung antar data.

Pada method main, dibuat Single Linked List awal dengan data 1 -> 2 -> 3 -> 4 -> 5 -> 6. Setelah itu, program mencetak list awal, kemudian menghapus node pertama sehingga data menjadi 2 -> 3 -> 4 -> 5 -> 6. Selanjutnya program menghapus node terakhir

sehingga data menjadi 2 -> 3 -> 4 -> 5. Terakhir, program menghapus node pada posisi ke-2 sehingga data menjadi 2 -> 4 -> 5.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Struktur Data dengan materi Single Linked List yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data dinamis yang terdiri dari beberapa node yang saling terhubung menggunakan pointer. Setiap node memiliki data dan link yang menunjuk ke node berikutnya sehingga data dapat tersusun secara berantai.

Dalam praktikum ini, penulis memahami cara membuat node, melakukan traversal, serta melakukan operasi dasar pada Single Linked List seperti penambahan node di depan, belakang, dan posisi tertentu, serta

penghapusan node pada bagian awal, akhir, maupun posisi tertentu. Selain itu, praktikum ini juga membantu memahami proses pencarian data pada linked list menggunakan proses traversal dari node pertama hingga node terakhir.

Melalui implementasi program menggunakan bahasa Java, dapat diketahui bahwa Single Linked List lebih fleksibel dibandingkan array karena ukuran data dapat berubah secara dinamis tanpa harus menentukan kapasitas di awal. Praktikum ini juga meningkatkan pemahaman mengenai penggunaan pointer atau reference dalam menghubungkan antar node pada struktur data linked list

3.2 Saran

Adapun saran yang dapat diberikan setelah pelaksanaan praktikum ini adalah sebagai berikut:

1. Mahasiswa diharapkan lebih sering berlatih membuat program Linked List agar lebih memahami penggunaan pointer dan hubungan antar node.
2. Praktikum selanjutnya sebaiknya diberikan contoh kasus yang lebih kompleks agar mahasiswa dapat memahami penerapan Linked List dalam penyelesaian masalah yang lebih nyata.
3. Mahasiswa disarankan untuk mempelajari jenis linked list lainnya seperti Double Linked List dan Circular Linked List untuk memperluas pemahaman mengenai struktur data linear dinamis.
4. Dalam pembuatan program, sebaiknya ditambahkan validasi agar program lebih aman dan dapat mengurangi kemungkinan terjadinya error.

DAFTAR PUSTAKA

[1] Binus University. "Single Linked List." Tersedia pada: https://socs.binus.ac.id/2017/03/15/single-linked-list/ . Diakses pada 6 Mei 2026.

[2] Politeknik Elektronika Negeri Surabaya. "Praktikum 15–16 Single Linked List." Tersedia pada: https://arna.lecturer.pens.ac.id/PrakASD_Java/Praktikum%2015-16%20-%20SingleLinkedList.pdf . Diakses pada 6 Mei 2026.

LAPORAN TUGAS PEKAN 5
STRUKTUR DATA

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. Kelas ADT Pasien(Pasien_2511531013)

```java
package pekan5_2511531013;

public class Pasien_2511531013 {

    // atribut data pasien
    private String namaPasien_1013;
    private String keluhan_1013;
    private int nomorAntrian_1013;

    // pointer ke pasien berikutnya
    private Pasien_2511531013 next_1013;

    // konstruktor untuk isi data pasien
    public Pasien_2511531013(String namaPasien_1013,String keluhan_1013,int nomorAntrian_1013) {
        this.namaPasien_1013 = namaPasien_1013;
        this.keluhan_1013 = keluhan_1013;
        this.nomorAntrian_1013 = nomorAntrian_1013;
        // node awal belum terhubung ke node lain
        this.next_1013 = null;
    }

    // getter nama pasien
    public String getNamaPasien_1013() {
        return namaPasien_1013;
    }

    // getter keluhan pasien
    public String getKeluhan_1013() {
        return keluhan_1013;
    }

    // getter nomor antrian
    public int getNomorAntrian_1013() {
        return nomorAntrian_1013;
    }

    // getter pointer next
    public Pasien_2511531013 getNext_1013() {
        return next_1013;
    }
```

```java
    // setter nama pasien
    public void setNamaPasien_1013(String namaPasien_1013) {
        this.namaPasien_1013 = namaPasien_1013;
    }

    // setter keluhan pasien
    public void setKeluhan_1013(String keluhan_1013) {
        this.keluhan_1013 = keluhan_1013;
    }

    // setter nomor antrian
    public void setNomorAntrian_1013(int nomorAntrian_1013) {
        this.nomorAntrian_1013 = nomorAntrian_1013;
    }

    // setter pointer next
    public void setNext_1013(Pasien_2511531013 next_1013) {
        this.next_1013 = next_1013;
    }

    // supaya data pasien bisa langsung dicetak
    @Override
    public String toString() {
        return "No Antrian : " + nomorAntrian_1013 + ", Nama : " + namaPasien_1013 + ", Keluhan : " + keluhan_1013;
    }
}
```

Pada kelas Pasien_2511531013, terdapat atribut namaPasien_1013, keluhan_1013, dan nomorAntrian_1013 yang digunakan untuk menyimpan data pasien. Selain itu, terdapat pointer next_1013 yang digunakan untuk menghubungkan node pasien dengan node berikutnya dalam linked list. Konstruktor digunakan untuk menginisialisasi seluruh atribut saat objek pasien dibuat. Program juga menyediakan method getter dan setter untuk mengambil serta mengubah nilai atribut. Method toString() dioverride agar data pasien dapat langsung ditampilkan dengan format yang lebih rapi.

2. Kelas Driver(RumahSakit_2511531013)

```java
package pekan5_2511531013;

import java.util.Scanner;

public class RumahSakit_2511531013 {

    // head linked list
    private Pasien_2511531013 head_1013;

    // menyimpan nomor antrian terakhir
    private int nomorTerakhir_1013;

    // menghitung jumlah pasien
    private int jumlahPasien_1013;

    // konstruktor
    public RumahSakit_2511531013() {
        head_1013 = null;
        nomorTerakhir_1013 = 0;
        jumlahPasien_1013 = 0;
    }

    // insert di tail / tambah pasien di akhir linked list
    public void daftarkanPasien_1013(String namaPasien_1013, String keluhan_1013) {
        // nomor antrian bertambah otomatis
        nomorTerakhir_1013++;
        // membuat node pasien baru
        Pasien_2511531013 pasienBaru_1013 = new Pasien_2511531013(namaPasien_1013, keluhan_1013, nomorTerakhir_1013);
        // jika linked list kosong
        if (head_1013 == null) {
            // pasien baru jadi head
            head_1013 = pasienBaru_1013;
        } else {
            // traversal sampai node terakhir
            Pasien_2511531013 curr_1013 = head_1013;
            while (curr_1013.getNext_1013() != null) {
                curr_1013 = curr_1013.getNext_1013();
            }
            // sambungkan node terakhir ke node baru
            curr_1013.setNext_1013(pasienBaru_1013);
        }

        jumlahPasien_1013++;
        System.out.println("Pasien berhasil didaftarkan!");
        System.out.println("Nomor Antrian : " + nomorTerakhir_1013);
    }
```

```java
    // delete head / panggil pasien terdepan
    public void panggilPasien_1013() {
        // cek apakah antrian kosong
        if (head_1013 == null) {
            System.out.println("Antrian kosong.");
            return;
        }

        // menyimpan pasien yang dipanggil
        Pasien_2511531013 pasienDipanggil_1013 = head_1013;
        // head digeser ke node berikutnya
        head_1013 = head_1013.getNext_1013();
        jumlahPasien_1013--;
        System.out.println("Pasien yang dipanggil:");
        System.out.println(pasienDipanggil_1013);
    }

    // menampilkan semua antrian pasien
    public void tampilkanAntrian_1013() {
        // cek apakah list kosong
        if (head_1013 == null) {
            System.out.println("Antrian masih kosong.");
            return;
        }

        Pasien_2511531013 curr_1013 = head_1013;

        int posisi_1013 = 1;

        System.out.println("Daftar Antrian Pasien:");

        // traversal linked list
        while (curr_1013 != null) {

            System.out.println("Posisi " + posisi_1013 + " -> " + curr_1013);
            curr_1013 = curr_1013.getNext_1013();
            posisi_1013++;
        }
    }
```

```java
    // mencari pasien berdasarkan nama
    public void cariPasien_1013(String namaCari_1013) {

        // cek apakah list kosong
        if (head_1013 == null) {

            System.out.println("Antrian kosong.");
            return;
        }

        Pasien_2511531013 curr_1013 = head_1013;
        int posisi_1013 = 1;
        boolean ditemukan_1013 = false;
        // traversal untuk pencarian data
        while (curr_1013 != null) {

            // pencarian tanpa membedakan huruf besar kecil
            if (curr_1013.getNamaPasien_1013().equalsIgnoreCase(namaCari_1013)) {
                System.out.println("Pasien ditemukan pada posisi " + posisi_1013);
                System.out.println(curr_1013);
                ditemukan_1013 = true;
                break;
            }
            curr_1013 = curr_1013.getNext_1013();
            posisi_1013++;
        }
        // jika data tidak ditemukan
        if (!ditemukan_1013) {
            System.out.println("Pasien dengan nama " + namaCari_1013 + " tidak ditemukan.");
        }
    }

    // cek status antrian
    public void cekStatusAntrian_1013() {
        // jika antrian kosong
        if (head_1013 == null) {
            System.out.println("Antrian kosong.");
        } else {
            System.out.println("Jumlah pasien : " + jumlahPasien_1013);
            System.out.println("Pasien terdepan : " + head_1013);
        }
    }

    // method untuk menampilkan menu
    public static void tampilkanMenu_1013() {
        System.out.println("\n=== Antrian Rumah Sakit 2511531013 ===");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan : ");
    }
    public static void main(String[] args) {
        Scanner input_1013 = new Scanner(System.in);
        RumahSakit_2511531013 rs_1013 = new RumahSakit_2511531013();
        int pilihan_1013;
        do {
            tampilkanMenu_1013();
            pilihan_1013 = input_1013.nextInt();
            input_1013.nextLine();
            switch (pilihan_1013) {
                case 1:
                    System.out.print("Masukkan Nama Pasien : ");
                    String namaPasien_1013 = input_1013.nextLine();
                    System.out.print("Masukkan Keluhan : ");
                    String keluhan_1013 = input_1013.nextLine();
                    rs_1013.daftarkanPasien_1013(namaPasien_1013, keluhan_1013);
                    break;
                case 2:
                    rs_1013.panggilPasien_1013();
                    break;
                case 3:
                    rs_1013.tampilkanAntrian_1013();
                    break;
                case 4:
                    System.out.print("Masukkan nama pasien yang dicari : ");
                    String namaCari_1013 = input_1013.nextLine();
                    rs_1013.cariPasien_1013(namaCari_1013);
                    break;
                case 5:
                    rs_1013.cekStatusAntrian_1013();
                    break;
                case 6:
                    System.out.println("Keluar dari program.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }

        } while (pilihan_1013 != 6);
        input_1013.close();
    }
}
```

Pada kelas RumahSakit_2511531013, terdapat pointer head_1013 yang menunjuk ke pasien pertama dalam antrian. Variabel nomorTerakhir_1013 digunakan untuk menyimpan nomor antrian terakhir agar nomor pasien bertambah secara otomatis, sedangkan jumlahPasien_1013 digunakan untuk menghitung jumlah pasien dalam antrian.

Method daftarkanPasien_1013() digunakan untuk menambahkan pasien baru di bagian akhir linked list (*insert at tail*). Jika linked list kosong, maka pasien baru langsung menjadi head. Jika tidak kosong, program akan melakukan traversal hingga node terakhir, kemudian node baru disambungkan ke akhir linked list.

Method panggilPasien_1013() digunakan untuk memanggil pasien terdepan dengan konsep *delete head*. Data pasien yang dipanggil akan ditampilkan, kemudian pointer head_1013 digeser ke node berikutnya sehingga pasien pertama keluar dari antrian.

Method tampilkanAntrian_1013() digunakan untuk menampilkan seluruh data pasien dalam antrian. Program melakukan traversal mulai dari head_1013 hingga node terakhir, lalu mencetak data setiap pasien beserta posisinya dalam antrian.

Method cariPasien_1013() digunakan untuk mencari pasien berdasarkan nama secara *case-insensitive*, sehingga huruf besar dan kecil tidak dibedakan. Program akan menelusuri linked list satu per satu hingga data ditemukan. Jika data tidak ditemukan, maka program akan menampilkan pesan bahwa pasien tidak ada dalam antrian.

Method cekStatusAntrian_1013() digunakan untuk menampilkan jumlah pasien yang sedang mengantri serta menampilkan data pasien yang berada di posisi paling depan. Jika linked list kosong, maka program akan menampilkan pesan bahwa antrian kosong.

Pada method main, program menggunakan perulangan do-while untuk menampilkan menu utama secara berulang hingga pengguna memilih keluar. Setiap menu diproses menggunakan switch-case, sehingga pengguna dapat

melakukan operasi seperti menambah pasien, memanggil pasien, menampilkan antrian, mencari pasien, dan mengecek status antrian.

3. Output

```
=== Antrian Rumah Sakit 2511531013 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : Budi Santoso
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan!
Nomor Antrian : 1

=== Antrian Rumah Sakit 2511531013 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 3
Daftar Antrian Pasien:
Posisi 1 -> No Antrian : 1, Nama : Budi Santoso, Keluhan : Demam

=== Antrian Rumah Sakit 2511531013 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 4
Masukkan nama pasien yang dicari : Budi Santoso
Pasien ditemukan pada posisi 1
No Antrian : 1, Nama : Budi Santoso, Keluhan : Demam

=== Antrian Rumah Sakit 2511531013 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 5
Jumlah pasien : 1
Pasien terdepan : No Antrian : 1, Nama : Budi Santoso, Keluhan : Demam

=== Antrian Rumah Sakit 2511531013 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 2
Pasien yang dipanggil:
No Antrian : 1, Nama : Budi Santoso, Keluhan : Demam

=== Antrian Rumah Sakit 2511531013 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 6
Keluar dari program.
```